

STUDENT ATTENDANCE SYSTEM

Project Report | Python Essentials

Prepared: 30 September 2026

1. Overview

The Student Attendance System is a local Python CLI tool for managing class rosters and attendance. It adapts the supplied Tkinter starter's core functions-student management, attendance marking, records, and below-threshold reports-into a modular design with persistent SQLite storage.

2. Problem statement

Paper or transient attendance is difficult to summarize and preserve. In the supplied starter, student records and statistics are recreated at startup while attendance is appended to CSV. This project provides persistent data and direct reports.

3. Objectives and scope

The project supports student and subject management, dated present/absent attendance, record listing, percentage summaries, and low-attendance reports. It is designed for a single local user. Login, multiple roles, a web service, GUI, and synchronization are out of scope.

4. Architecture

The package separates database setup (`database.py`), persistence (`repository.py`), session processing (`processing.py`), reporting (`analytics.py`), and CLI interaction (`cli.py`). Tests are in `tests/test_attendance.py`. SQLite provides persistence and relational constraints; there are no third-party runtime dependencies.

5. Functional requirements

1. Add, list, and remove students.
2. Add and list subjects.
3. Record one status per student, subject, and date.
4. List attendance records, optionally by subject.
5. Calculate attendance percentages and filter below a threshold.

6. Non-functional requirements

- **Error handling:** validate names, ISO dates, status values, thresholds, and duplicates; show concise CLI errors.
- **Logging:** unexpected failures include diagnostics when `--verbose` is used.
- **Performance:** an index supports subject/date access; reports use SQL aggregation.
- **Maintainability:** independent modules, parameterized SQL, and automated unit tests.

- **Data integrity:** SQLite foreign keys, uniqueness constraints, status checks, and transaction rollback protect consistency.

7. Usage

Initialize with `python -m attendance_system.cli init`. Add a student and subject, then use their printed IDs with `mark`; `report` and `below-threshold` provide analytics. See [README.md](#) for commands.

8. Testing

Run `python -m unittest discover -v` for tests of CRUD validation, attendance calculations, threshold filtering, and rollback. All four included unit tests were executed successfully with Python 3.12.

9. VITyarthi guideline alignment

The project follows the request's requirements: modular Python, CRUD, processing, analytics/reporting, non-functional requirements, CLI execution, root README and statement, and unit tests. No separate VITyarthi Python Essentials guideline document was provided, so additional course-specific requirements cannot be confirmed.

10. Limitations and future work

The application targets local single-user use. Possible extensions include CSV export, editing attendance, a GUI, and authenticated multi-user access.